

Report

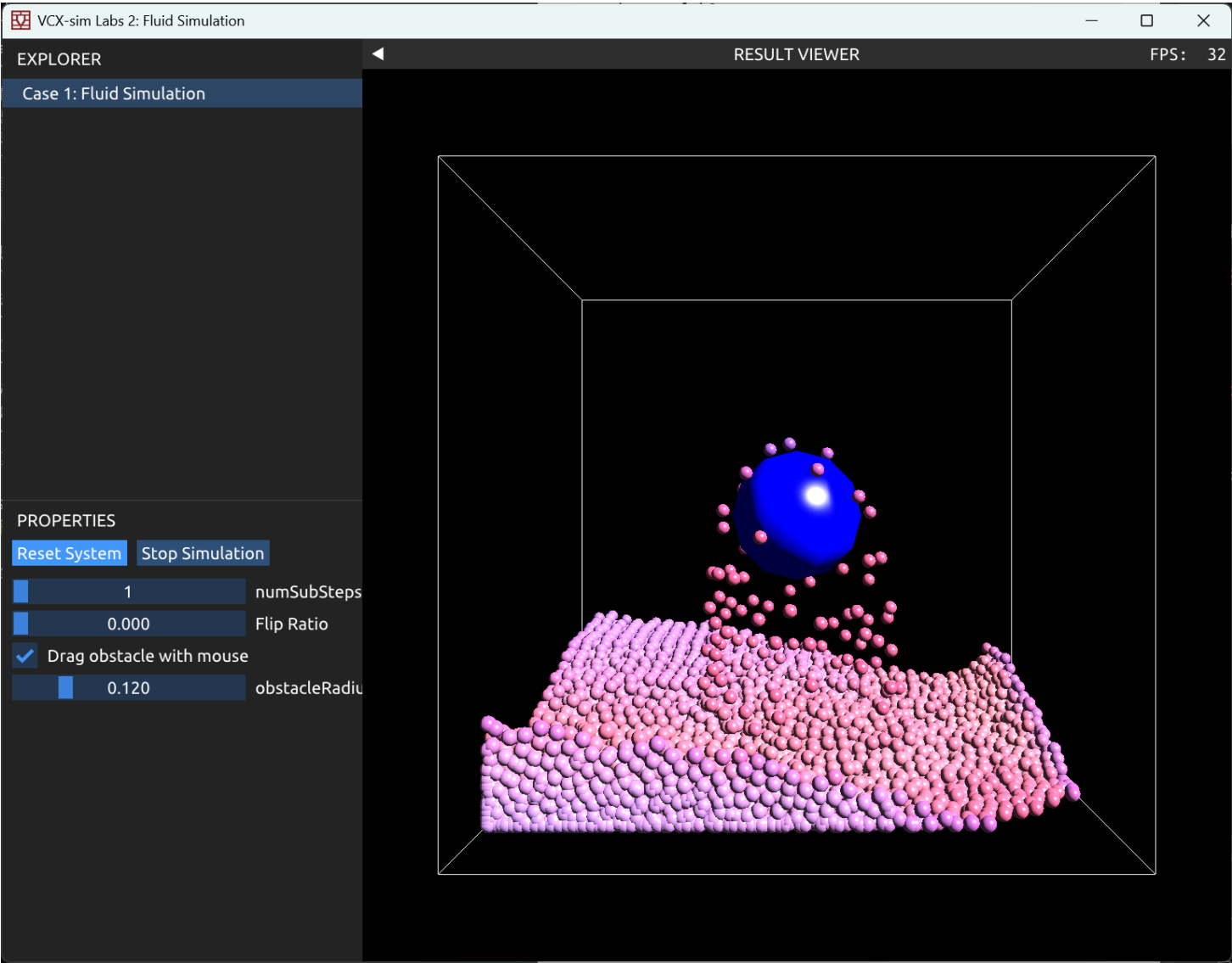
结果

运行

```
xmake run lab2
```

查看。页面内设置了Flip Ratio等参数供调整。实现了依据粒子的速度（也包含密度等，但影响太小了）进行不同的染色以及拖动球形障碍物与场景交互的功能。

以下分别是场景开始时的渲染效果，以及转动视角后拖动球在流体里快速搅动的渲染效果：



实现思路

按照给出的循环框架填写对应的函数，其中：

- `integrateParticles`：对每个粒子叠加重力并用当前速度推进位置。
- `handleParticleCollisions`：对粒子与边界、球形障碍物做位置约束，并用障碍物相对速度修正碰撞后的粒子速度。
- `pushParticlesApart`：按照要求用空间哈希把粒子按网格单元分类，再检查邻域内的粒子对并做位置修正，同时把修正量折算成速度增量回写。
- `ResetSystem`：重新初始化流体粒子、网格和障碍物状态，保证每次重置后仿真从同一初始条件开始。

Bonus说明

在`Simulator::updateParticleColors()`中实现了基于速度大小和密度对颜色插值的效果：先在粒子位置采样网格压强和密度，再结合粒子速度做红蓝插值。最终的效果是速度越快，颜色越红，速度越慢颜色越蓝。在渲染之初，流体从高处流下的过程中，粒子由粉红变蓝紫色。